Assignment No. 2

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git

---

Quick Sort for Personalized Movie Recommendation System

1. Problem Statement

A popular OTT platform, StreamFlix, provides personalized movie recommendations by sorting movies according to user preferences such as IMDb rating, release year, or watch-time popularity. During peak hours, the platform may need to process and sort a large number of movie records, which can affect response time.

As a backend engineer, the objective is to:

- Implement the Quick Sort algorithm for efficiently sorting movie records.
- Allow movies to be sorted according to different user-selected parameters such as IMDb rating, release year, and watch time.
- Handle large movie datasets while maintaining efficient response times.
- Analyze the time and space complexity of the implemented algorithm.
- Demonstrate the suitability of the Divide and Conquer strategy for this real-world application.

---

2. Course Objective

The objectives of this assignment are:

1. To understand the basics of **computational complexity** of algorithms.
2. To analyze the performance of sorting algorithms.
3. To select an appropriate algorithm design strategy for solving real-world problems.
4. To understand and implement the **Divide and Conquer** paradigm.
5. To apply Quick Sort to a practical movie recommendation scenario.

---

3. Course Outcome

After completing this assignment, the student will be able to:

- Analyze the **asymptotic performance** of algorithms.
- Apply suitable algorithmic paradigms to computational problems.
- Implement and analyze the **Quick Sort algorithm**.
- Understand the practical application of Divide and Conquer.
- Select an appropriate sorting technique for large datasets.

---

4. Theory

4.1 Quick Sort

Quick Sort is a widely used comparison-based sorting algorithm based on the **Divide and Conquer** paradigm. It is particularly useful when large datasets need to be sorted efficiently.

Quick Sort works by selecting an element called a **pivot** and rearranging the array around it. Elements smaller than the pivot are placed on one side, while elements greater than the pivot are placed on the other side. The same process is then recursively applied to the two resulting subarrays.

Quick Sort is generally an **in-place sorting algorithm**, because the partitioning can be performed within the original array without requiring another array of size n.

4.2 Working of Quick Sort

Quick Sort consists of three main steps:

1. Divide

A pivot element is selected from the array.

Common pivot-selection strategies include:

- First element
- Last element
- Random element
- Median-of-three

In this implementation, the **last element is selected as the pivot**.

2. Partition

The array is rearranged around the pivot.

- Elements smaller than or equal to the pivot are placed before it.
- Elements greater than the pivot are placed after it.
- The pivot is placed in its final sorted position.

3. Conquer

Quick Sort is recursively applied to:

- The subarray to the left of the pivot.
- The subarray to the right of the pivot.

The process continues until each subarray contains zero or one element.

4. Combine

Unlike Merge Sort, Quick Sort does not require a separate combine operation. Once all recursive calls are completed, the complete array is already sorted.

---

5. Quick Sort Algorithm

Algorithm: QuickSort

```
ALGORITHM QuickSort(arr, low, high)

1. IF low < high THEN
2.     pivotIndex ← Partition(arr, low, high)
3.     QuickSort(arr, low, pivotIndex - 1)
4.     QuickSort(arr, pivotIndex + 1, high)
5. END IF

END ALGORITHM
Algorithm: Partition
ALGORITHM Partition(arr, low, high)

1. pivot ← arr[high]
2. i ← low - 1

3. FOR j ← low TO high - 1 DO
4.     IF arr[j] ≤ pivot THEN
5.         i ← i + 1
6.         SWAP(arr[i], arr[j])
7.     END IF
```

```
8. END FOR

9. SWAP(arr[i + 1], arr[high])

10. RETURN i + 1

END ALGORITHM
```

---

6. Example

Consider the following array:

[3, 1, 4, 2]

Step 1: Select Pivot

The last element is selected as the pivot.

Pivot = 2

Step 2: Partition

Compare each element with the pivot:

3 > 2
1 < 2
4 > 2

After partitioning:

[1, 2, 4, 3]
  ↑
Pivot

The pivot 2 is now in its correct sorted position.

Step 3: Recursive Calls

The array is divided into:

Left subarray = [1]
Right subarray = [4, 3]

The left subarray contains only one element and is already sorted.

For `[4, 3]`, select the last element:

Pivot = 3

After partitioning:

[3, 4]

Final Sorted Array

[1, 2, 3, 4]

---

7. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	O(n log n)	Pivot divides the array into approximately equal parts
Average Case	O(n log n)	Partitions are reasonably balanced
Worst Case	$O(n^2)$	Pivot repeatedly becomes the smallest or largest element
Space – Best/Average	O(log n)	Recursion stack for balanced partitions
Space – Worst	O(n)	Recursion stack for highly unbalanced partitions

Explanation

The best and average time complexity of Quick Sort is: **O(n log n)**

This occurs when the pivot divides the array into reasonably balanced subarrays.

The worst-case complexity is: **$O(n^2)$**

This can occur when the pivot repeatedly produces highly unbalanced partitions, such as when the input is already sorted and the first or last element is consistently selected as the pivot.

The auxiliary space complexity is primarily determined by the recursion stack:

Best/Average Case → **O(log n)**

Worst Case → **O(n)**

---

8. Optimizations for Quick Sort

Quick Sort can be optimized for large datasets using the following techniques:

8.1 Random Pivot Selection

Selecting a random pivot reduces the probability of repeatedly obtaining highly unbalanced partitions.

8.2 Median-of-Three

The median of the first, middle, and last elements can be selected as the pivot. This often produces better partitions for partially sorted data.

8.3 Tail Recursion Optimization

The larger subarray can be processed iteratively while recursion is applied only to the smaller subarray. This reduces the maximum recursion depth.

8.4 Hybrid Quick Sort

For very small subarrays, **Insertion Sort** can be used instead of Quick Sort because its low overhead can make it faster for small inputs.

---

9. StreamFlix Movie Dataset

Each movie can be represented using the following attributes:

Attribute	Description
Movie ID	Unique identifier of the movie
Movie Name	Name of the movie
IMDb Rating	IMDb rating of the movie
Release Year	Year in which the movie was released
Watch Time	Total watch time/popularity measure

For example:

Movie ID	Movie Name	IMDb Rating	Release Year	Watch Time
101	Movie A	8.2	2020	150000
102	Movie B	6.5	2018	95000
103	Movie C	9.0	2022	240000
104	Movie D	7.3	2019	125000
105	Movie E	8.7	2021	200000

The system can sort these records according to the parameter selected by the user.

---

10. Implementation

The source code and assignment files are uploaded to the GitHub repository.

```
Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git
```

11. Conclusion

Quick Sort is an efficient sorting algorithm based on the **Divide and Conquer** paradigm. In the StreamFlix application, it can be used to sort movies according to user-selected parameters such as IMDb rating, release year, and watch-time popularity.

The algorithm provides an average time complexity of **O(n log n)** and requires relatively little additional memory. However, poor pivot selection can result in a worst-case complexity of **$O(n^2)$**. Appropriate pivot-selection and optimization techniques can reduce the likelihood of poor performance.

Therefore, Quick Sort is a suitable algorithm for efficiently organizing large movie datasets and can contribute to responsive sorting and recommendation operations in an OTT platform such as StreamFlix.
